Longest Substring Without Repeating Characters

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Problem Statement

Write a C program that reads a string s and passes its address to a function named longest_unique_substring(). The function must find the longest substring of s that contains no repeated characters and display the length of that substring.

A substring is a continuous part of a string. For example, "abc" is a substring of "abcabcbb", but "acb" is not, because its characters are not next to each other in the original string.

2. Requirements

- Read a string from the user.
- Pass the string to the function longest_unique_substring() using its address (pointer).
- Use pointers to pass the string between the calling function (main) and longest_unique_substring().
- Find the longest substring with no repeated characters and display its length.
- Treat uppercase and lowercase letters as different characters (so "a" and "A" are different).
- Display 0 if the input string is empty.

3. Concepts Used

- **Strings and character arrays:** the input is stored in a char array and ends with the null character '\0'.
- **Pointers:** the function receives a char * that holds the address of the first character of the string.
- **Pointer arithmetic:** pointers are moved forward (p++) and subtracted (p - start) to find the length of a window.
- **Sliding window technique:** a "window" with a start pointer and an end pointer moves over the string and always contains unique characters.
- **Array of pointers:** last_seen[256] stores the address where every character was last seen.
- **Functions, loops and conditions:** for loop, if statement and a void function with a pointer parameter.
- **fgets():** reads a full line (including spaces) safely without overflowing the array.

4. Algorithm

1. Start the program.
2. Read the string using fgets() and remove the trailing newline character.
3. Call longest_unique_substring(s), passing the address of the first character of the string.
4. Inside the function, set start = s and max_len = 0, and set every entry of last_seen[] to NULL.

5. Move a pointer p from the first character to the end of the string. For each character ch at p:

   (a) If ch was seen before and its last address is at or after start, move start to one position after that address.

   (b) Store the current address p in last_seen[ch].

   (c) Calculate the window length as (p - start) + 1. If it is greater than max_len, update max_len.

6. After the loop ends, display max_len. For an empty string the loop never runs, so 0 is displayed.
7. Stop.

Dry Run for the string "abcabcbb"

The table shows how the window changes when each character is processed.

Index	Character	Window (start to p)	Window Length	max_len
0	a	a	1	1
1	b	ab	2	2
2	c	abc	3	3
3	a	bca	3	3
4	b	cab	3	3
5	c	abc	3	3
6	b	cb	2	3
7	b	b	1	3

Final answer: the length of the longest substring without repeating characters is 3.

5. Source Code

File name: longest_unique_substring.c

```
/*
 * Program : Longest Substring Without Repeating Characters
 * Name    : Rudra Pratap Singh
 * SAP ID  : 590032811
 * Batch   : 16
 */

#include <stdio.h>
#include <string.h>

#define MAX_LEN 1001   /* maximum input length (1000 characters + '\0') */
#define CHAR_SET 256   /* total possible character values (ASCII)        */

/*
 * Function : longest_unique_substring
 * Input    : s - pointer to the first character of the string
 * Task     : Finds the length of the longest substring of s that has
 *            no repeated characters and displays it.
 *            Uppercase and lowercase letters are treated as different.
 *            Displays 0 if the string is empty.
 */
void longest_unique_substring(char *s)
{
    char *last_seen[CHAR_SET]; /* address where each character was last seen */
    char *start = s;           /* pointer to start of the current window     */
    char *p;                   /* pointer that moves over the string         */
    int max_len = 0;           /* length of the longest substring found      */
    int cur_len;
    int i;

    /* Initially no character has been seen */
    for (i = 0; i < CHAR_SET; i++)
        last_seen[i] = NULL;

    for (p = s; *p != '\0'; p++)
    {
        unsigned char ch = (unsigned char)*p;

        /* If ch already appears inside the current window,
           move the window start just after its previous position */
        if (last_seen[ch] != NULL && last_seen[ch] >= start)
            start = last_seen[ch] + 1;

        last_seen[ch] = p;           /* remember latest position of ch */

        cur_len = (int)(p - start) + 1;  /* length of current window   */
        if (cur_len > max_len)
            max_len = cur_len;
    }

    printf("Length of longest substring: %d\n", max_len);
}
```

```
int main(void)
{
    char s[MAX LEN];
    size_t n;

    printf("Enter a string: ");

    /* fgets reads the whole line, including spaces */
    if (fgets(s, sizeof(s), stdin) == NULL)
        s[0] = '\0';                /* no input -> treat as empty string */

    /* Remove the trailing newline character, if present */
    n = strlen(s);
    if (n > 0 && s[n - 1] == '\n')
        s[n - 1] = '\0';

    /* Pass the address of the string to the function */
    longest_unique_substring(s);

    return 0;
}
```

6. Test Cases and Output

The program was compiled with gcc and tested with the four given examples and some extra cases. The results are shown below.

No.	Input String	Longest Substring	Output	Remark
1	`abcabcbb`	abc	3	Given example 1
2	`bbbbb`	b	1	Given example 2
3	`pwwkew`	wke	3	Given example 3
4	`(empty string)`	(none)	0	Given example 4: empty input
5	`AaBb`	AaBb	4	Uppercase and lowercase are different
6	`dvdf`	vdf	3	Window moves past the repeated d
7	`abba`	ab or ba	2	Old repeat is ignored correctly
8	`a b a c`	a b (or a c)	3	Space is treated as a character

Program Output

Run 1

```
Enter a string: abcabcbb
Length of longest substring: 3
```

Run 2

```
Enter a string: bbbbb
Length of longest substring: 1
```

Run 3

```
Enter a string: pwwkew
Length of longest substring: 3
```

Run 4 (empty string)

```
Enter a string:
Length of longest substring: 0
```

Run 5

```
Enter a string: AaBb
Length of longest substring: 4
```

7. Explanation and Complexity

The simple method would check every possible substring, which is slow for long strings. This program uses the sliding window technique, so the string is scanned only once from left to right.

The array last_seen[] remembers the address of the latest occurrence of every character. When a character repeats inside the current window, the start pointer jumps directly to the position after its previous occurrence, so the window never contains a repeated character. Because the characters are stored by their ASCII value and not by letter name, 'a' and 'A' have different entries, which makes the program case sensitive as required.

Measure	Value	Reason
Time complexity	O(n)	Each character is visited exactly once.
Space complexity	O(1)	last_seen[] has a fixed size of 256 entries.

8. Conclusion

The program successfully finds the length of the longest substring without repeating characters. The string is passed to longest_unique_substring() as a pointer, pointer arithmetic is used to calculate lengths, uppercase and lowercase letters are treated as different characters, and an empty string correctly gives 0. All the given examples produced the expected output. This project helped in understanding strings, pointers, pointer arithmetic and the sliding window technique in C.